HARNESS LAB · MÓDULO 5 · AULA 5.3

Avisa quando termina *ou trava*

O loop agora é código, então ele pode contar o que está fazendo. O aviso de dentro conta o progresso; o de fora conta quando ele parou. Com os dois, você põe para rodar e vai para a academia.

`Evolution API` `.claude/scripts/.env` `RALPH_NOTIFY` `aviso.sh` `aviso-env.sh` `vigia.sh` `run.tsv` `AVISO_DRY=1` `AVISO_CMD` `ENSAIO-AVISO.md`

VOCÊ SAI COM

O ralph da aula mandando WhatsApp a cada etapa, o aviso de fora (fim do `loop.sh` e vigia) no canal que você escolher, o token fora do repositório e o primeiro run de uma task que parou sozinho e avisou no celular.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

Como o loop virou um código determinístico que gerencia o que vai ser construído, ele pode avisar o que está fazendo: o que começou, o que concluiu, o que vem depois. O Lucas tem uma instância da **Evolution API** no servidor dele, com o próprio número conectado, e quer que o loop mande WhatsApp para ele mesmo: "vou iniciar a fase 1", "terminei a fase 1". Assim dá para pôr para rodar, ir para a academia e voltar sabendo o que aconteceu. O canal é o de menos: bot do Telegram, Slack, Discord, o que você preferir.

Nada foi codado na mão. Num `.env` ao lado do `ralph.sh`, em `.claude/scripts/`, ficaram o token da API, o nome da instância e a URL. Depois, este pedido ao Claude Code:

O PEDIDO DA AULA (ACENTOS CORRIGIDOS)

```
Quero que você inclua no @.claude/scripts/ralph.sh envio de mensagem utilizando a Evolution API
para mandar uma mensagem para mim mesmo. Utilize as variáveis de ambiente que estão no mesmo
nível do ralph.sh.

Quero mensagem para o início de uma feature, início de uma phase, se os gates passaram e por fim
o final de uma fase. E no final a feature completa.
```

O que voltou, evento por evento:

EVENTO	O QUE A MENSAGEM TRAZ
Início da feature	Feature, repositório, fases pendentes do total, engine
Início de fase	Posição (1/2, 2/2) e o título da fase
Gates verdes	Ciclo X/N e quais gates rodaram (0, 2 e 3), com o que foi pulado
Gate vermelho	Ciclo, qual gate falhou e as primeiras 8 linhas da causa. Não estava no pedido: o Claude incluiu e avisou
Fim de fase	Hash do commit, duração e ciclos (ou "já estava implementada"); falha, se falhar
Fim da feature	COMPLETA ou com falhas: contagens, duração e as fases que falharam

Saber pedir é o trabalho

"Hoje o mais importante é a gente saber pedir." O pedido diz o arquivo (`@`), o canal, onde estão os segredos e a lista de eventos. Nenhuma linha de código foi escrita à mão.

Leia o que voltou

Lê o `.env` sem executar, falha de envio não derruba o run, `RALPH_NOTIFY=off` desliga. E um limite: "se o run aborta pelo limite de esperas de uso, a mensagem de fim da feature não é enviada".

8 mensagens num run verde de 2 fases (2 da feature e 3 por fase), medidas com o ralph da aula, um `claude` falso e uma Evolution falsa. Com 5 fases verdes de primeira, 17.

2 Indo *além da aula*

ALÉM DA AULA · O RALPH SÓ AVISA O QUE ELE VIVE PARA VER

As mensagens saem de dentro do ralph. Se ele morre (`kill -9`, terminal fechado, máquina reiniciada), se sai pelo limite de esperas de uso, ou se uma sessão pendura sem nunca terminar, o celular fica em silêncio. E silêncio parece "está rodando". O ralph não tem timeout de sessão nem confere se está vivo. O aviso de que ele **parou** tem de sair de fora:

- o `loop.sh` do curso avisa no fim, pelo exit code e pelo `.phases/state/run.tsv`: terminou, falhou (com fase, gate e causa) ou interrompido (o Ctrl-C também avisa);
- o `vigia.sh` lê o `run.tsv` a cada 60 s e avisa uma vez por evento: esperando limite de uso, PID do ralph morto com status `running`, log da sessão parado há mais de 15 min ("travou?").

Um detalhe que decide qual usar: o `run.tsv` é escrito pelo ralph do plugin bc-harness 0.2.0, o que o `loop.sh` chama. O ralph da aula é a cópia local de julho, sem esse arquivo: com ele, o aviso é só o de dentro.

Fonte: código dos dois ralphs e do `vigia.sh`; vigia testado com um `run.tsv` falso (PID morto e log parado).

ALÉM DA AULA · HOOK DE STOP NÃO QUER DIZER "TERMINOU"

Parece o lugar óbvio para o aviso, e é o errado. Cada fase abre duas sessões (implementação e juiz), e os gates rodam depois que a sessão sai. Um Stop avisaria "terminei" de uma fase que ainda vai reprovar no teste. Nenhuma variável `RALPH_*` diz se a sessão é a do juiz.

> *"Stop hooks fire whenever Claude finishes responding, not only at task completion."*
>
> Documentação oficial do Claude Code · Automate actions with hooks, "Limitations" (code.claude.com/docs/en/hooks-guide)

ALÉM DA AULA · QUAL CANAL, E O QUE CADA UM COBRA

Evolution API não é oficial: no modo Baileys ela usa o WhatsApp Web, e o número pode ser bloqueado por automação. A doc do Baileys diz: "We discourage any stalkerware, bulk or automated messaging usage." O **WhatsApp oficial** (Cloud API) cobra por mensagem entregue desde 01/07/2025 e, fora da janela de 24 h, só aceita template aprovado. **Telegram** é grátis, sem aprovação, e o bot sai do @BotFather com `/newbot`, mas "Bots can't start conversations with users": mande `/start` ao seu bot antes. No **Slack**, a URL do webhook é o segredo. No **Discord**, o webhook tem um endpoint compatível com o Slack: a URL com `/slack` no fim recebe o mesmo `{"text": ...}`, então ela vai na variável do Slack, sem mudar uma linha.

Fontes: github.com/WhiskeySockets/Baileys; developers.facebook.com/docs/whatsapp/pricing; core.telegram.org/bots/features; discord.com/developers/docs/resources/webhook, "Execute Slack-Compatible Webhook". Discord testado num receptor local.

ALÉM DA AULA · O TOKEN FORA DO REPOSITÓRIO E FORA DO TERMINAL

O `.env` ao lado do `ralph.sh` fica dentro do repositório. Sem estar ignorado, ele suja a árvore e o ralph se recusa a rodar ("Arvore de trabalho suja"); num script com `git add -A`, iria para o commit. A aula instala um `.claude/scripts/.gitignore` com `.env`, e o passo 4 vai além: o arquivo de verdade fica em `~/.config/harness-lab/aviso.env`, com `chmod 600`, e no projeto entra só um link.

Não exporte o token no terminal. Variável exportada passa para todo processo filho: o ralph, cada `claude -p` do loop e o Bash que a sessão abre. O ralph da aula e o `aviso-env.sh` leem o arquivo sem exportar. Na Evolution, a `AUTHENTICATION_API_KEY` é a chave global do servidor; o envio aceita também o token da instância, que só manda mensagem por ela. Prefira o token e preencha o `EVOLUTION_NUMBER`. Com a 2.3 instalada, o `protege-env.sh` barra Read, Edit e Write no link `.claude/scripts/.env`; o `aviso.env` em `~/.config` e um `cat` pelo Bash passam. O que protege o token ali é ele não estar no repositório nem no ambiente.

Fontes: docs.evolutionfoundation.com.br, "Variáveis de Ambiente" ("Chave da API usada para autenticação global") e "Send Text Message" ("global or instance-specific"). O ralph recusando o `.env` fora do `.gitignore` foi testado; o `protege-env.sh` da 2.3 casa só os nomes `.env` e `.env.*`.

ALÉM DA AULA · ENSAIE O AVISO ANTES DE PRECISAR DELE

`AVISO_DRY=1` mostra o curl que sairia, com o nome das variáveis no lugar dos segredos. O primeiro run de verdade é um **ENSAIO** de uma fase com uma task trivial, numa branch descartável: custa uma sessão e um juiz se passar de primeira (o pior caso, que o `--checa` mostra, é 6 sessões com `MAX_CICLOS=3`), e é a primeira vez que o shim, o `--max-budget-usd`, o vigia e o aviso rodam com o CLI real. Melhor descobrir ali que o celular não toca do que no meio da feature-alvo.

Passos 5 a 7 da seção 3. Pedido do trap de saída e da versão com menos mensagens: `modelos/pedidos-do-aviso.md`.

3 Instale e faça *no seu repositório*

Precisa de Linux ou WSL (macOS não testado), bash 4 ou mais novo, `git`, `curl` e o `timeout` do GNU coreutils. O `loop.sh` usa o ralph do plugin bc-harness (aula 3.4) e o hook `guarda-aceite.py` da aula 4.2 (`instalar.sh 4.2`) com o seu `.claude/aceite.txt`.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize com `git -C ~/harness-lab-materiais pull`.

2 O bc-harness, se você ainda não instalou na 3.4.

É dele o ralph que o `loop.sh` chama (o que escreve o `run.tsv`). Instala para o seu usuário, em todos os projetos.

Terminal — claude

```
claude plugin marketplace add beerandcodeteam/beer-and-code-harness
claude plugin install bc-harness@beer-and-code
```

3 Instale os arquivos da aula e commite.

Em `.claude/scripts/`: o `ralph.sh` da gravação, com o WhatsApp dentro (o `loop.sh` segue chamando o do plugin), o `aviso-env.sh`, o `.gitignore` do `.env` e, iguais aos da 5.1 e da 5.2 ("igual", se já tem), o `loop.sh`, o `shim/claude`, o `aviso.sh`, o `vigia.sh` e o `confere-phases.sh`.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 5.3
```

Conflito no `ralph.sh`? Você já tem um. Fez o pedido nele? Fique com ele e apague o `ralph.sh.harness-lab-novo`. Não fez? Troque o seu por ele (`mv`). Depois, os scripts entram no git: o `loop.sh` exige árvore limpa.

Terminal · na raiz do seu projeto — shell

```
git add .claude/scripts
git commit -m "harness: aviso do loop (aula 5.3)"
```

4 Guarde o token fora do repositório.

Cria `~/.config/harness-lab/aviso.env` a partir do modelo, fecha a permissão e liga no projeto com um link. Já criou o `.env` como na aula? Ele vira o `aviso.env`, não se perde. Nada é sobrescrito: um `aviso.env` ou um link que já existam ficam como estão. O último comando prova que o git ignora o link: o esperado é `.claude/scripts/.gitignore:3:.env`.

Terminal · na raiz do seu projeto — shell

```
A=$(echo ~/harness-lab-materiais/modulo-5-loop/aula-5.3-*)
E=~/.config/harness-lab/aviso.env
L=.claude/scripts/.env
mkdir -p ~/.config/harness-lab
[ -L "$L" ] || [ ! -f "$L" ] || [ -f "$E" ] || mv "$L" "$E"
[ -f "$E" ] || cp "$A/modelos/aviso.env.exemplo" "$E"
chmod 600 "$E"
[ -e "$L" ] || ln -sfn "$E" "$L"
git check-ignore -v "$L"
```

Abra o `~/.config/harness-lab/aviso.env` no seu editor e preencha só o canal que você usa. Para o WhatsApp da aula: `EVOLUTION_API_URL`, `AUTHENTICATION_API_KEY`, `EVOLUTION_INSTANCE` e, de preferência, `EVOLUTION_NUMBER`. Telegram: `TELEGRAM_TOKEN` e `TELEGRAM_CHAT_ID`. Slack ou Discord: `SLACK_WEBHOOK_URL`. Em outro projeto, rode o mesmo bloco: com o `aviso.env` já criado, ele só faz o link (um `.env` próprio daquele projeto fica como está).

5

Teste o canal: primeiro sem mandar, depois de verdade.

O `aviso-env.sh` lê o arquivo e escolhe o canal: WhatsApp se a Evolution está preenchida, senão Telegram, Slack (ou Discord), webhook, e por fim notificação local. Com `AVISO_DRY=1` ele só mostra o curl. Sem canal preenchido, a resposta é `local: teste do loop`.

Terminal · na raiz do seu projeto — shell

```
AVISO_DRY=1 .claude/scripts/aviso-env.sh "teste do loop"
.claude/scripts/aviso-env.sh "teste do loop: chegou?"
```

Chegou no celular? Se o seu canal é o WhatsApp, o ralph da aula vai chegar também: ele lê o mesmo `.claude/scripts/.env` e chama o mesmo endpoint da Evolution. Com Telegram, Slack ou Discord, o ralph da aula não manda nada: peça a versão do seu canal com o pedido 2 de `modelos/pedidos-do-aviso.md`. É um `curl` no script, não uma sessão: o aviso não gasta token. Erro de envio aparece no terminal e o script segue, de propósito.

.claude/scripts/ralph.sh · notify(), resumido — instalado pelo instalar.sh · não copie do PDF

```
[ "$NOTIFY_MODE" = "off" ] && return 0              # RALPH_NOTIFY=off
payload="{\"number\":\"$NOTIFY_NUMBER\",\"text\":\"$text\"}"   # com escape de JSON
http_code=$(curl -sS -m 15 -o /dev/null -w '%{http_code}' \
  -X POST "$NOTIFY_URL/message/sendText/$NOTIFY_INSTANCE" \
  -H "apikey: $NOTIFY_KEY" -H 'Content-Type: application/json' \
  -d "$payload" 2>/dev/null || true)
[[ "$http_code" =~ ^2 ]] || warn "Falha ao enviar notificacao WhatsApp (HTTP $http_code)"
return 0                                    # falha no envio nunca derruba o run
```

6 Veja o vigia avisar, sem gastar token.

O `modelos/run-morto.tsv` é um `run.tsv` de mentira: status `running` e um PID que não existe. O vigia lê, avisa e sai. O esperado: `local: loop demo: morto? o ralph (PID 4194304) sumiu com status running na fase 1.`

Terminal · na raiz do seu projeto — shell

```
A=$(echo ~/harness-lab-materiais/modulo-5-loop/aula-5.3-*)
D=$(mktemp -d)
mkdir -p "$D/.phases/state"
cp "$A/modelos/run-morto.tsv" "$D/.phases/state/run.tsv"
LOOP_INICIO=0 AVISO_DRY=1 "$PWD/.claude/scripts/vigia.sh" "$D"
```

7 O primeiro run de verdade: um ENSAIO de uma task.

Numa branch descartável, com o modelo da aula. Antes do commit, troque no `ENSAIO-AVISO.md` o caminho e o teste pelos do seu projeto (o modelo é Laravel com Pest: um teste de fumaça de `/eventos`). O `AVISO_CMD` faz o `loop.sh` e o vigia mandarem pelo `aviso-env.sh`; sem ele, o `loop.sh` chama o `aviso.sh`, que só enxerga variável exportada. O `--checa` confere tudo sem gastar token (ele mostra `aviso: local (AVISO_CMD)`: normal, quem escolhe o canal é o `aviso-env.sh`).

Terminal · na raiz do seu projeto — shell

```
A=$(echo ~/harness-lab-materiais/modulo-5-loop/aula-5.3-*)
git switch -c ensaio-aviso
cp "$A/modelos/ENSAIO-AVISO.md" .
```

Terminal · na raiz do seu projeto · depois de editar a task — shell

```
git add ENSAIO-AVISO.md
git commit -m "ensaio do aviso"
export AVISO_CMD=.claude/scripts/aviso-env.sh
.claude/scripts/loop.sh --checa ENSAIO-AVISO.md
```

Pré-voo ok, rode de verdade. Custa uma sessão e um juiz se passar de primeira; o pior caso é 6 sessões (`MAX_CICLOS=3`), como mostra o `--checa`. Enquanto roda, está rodando ali na sua frente; "funcionou" só quando chegar `loop <projeto>: terminou 1/1 fases`.

Terminal · seu projeto real · gasta de 2 a 6 sessões — loop.sh

```
.claude/scripts/loop.sh ENSAIO-AVISO.md
```

Ou, para o progresso fase a fase da aula, o mesmo ENSAIO pelo ralph da aula: 5 mensagens no WhatsApp. Escolha um dos dois, porque cada run custa. Pelo ralph da aula não passam o shim (`LOOP_TIMEOUT` e `--max-budget-usd`), o pré-voo nem o vigia: é o progresso no celular sem os freios da 5.2. Para a feature-alvo (6.1), use o `loop.sh`.

Terminal · seu projeto real · gasta de 2 a 6 sessões, sem o shim — ralph.sh

```
.claude/scripts/ralph.sh --engine claude ENSAIO-AVISO.md
```

Chegou o aviso? Volte para a sua branch e apague a do ensaio:

Terminal · na raiz do seu projeto — shell

```
git switch -
git branch -D ensaio-aviso
```

4 Arquivos que *reforçam a aula*

`.claude/scripts/ralph.sh`

O ralph atualizado da gravação: o `ralph.sh` do bc-harness de julho (commit `e6ef269`) mais as 186 linhas que o Claude escreveu na aula. Testado com um `claude` falso: 8 mensagens num run de 2 fases, nenhuma com `RALPH_NOTIFY=off`.

No repositório: modulo-5-loop/aula-5.3-avisa-quando-termina-ou-trava/projeto/

`vigia.sh, loop.sh e aviso-env.sh`

O aviso de fora: o vigia avisa travou, morto e limite de uso; o `loop.sh` avisa o fim do run (no código, o bloco `[5.3]`); o `aviso-env.sh` leva o WhatsApp da aula para os dois.

No repositório: modulo-5-loop/aula-5.3-avisa-quando-termina-ou-trava/projeto/.claude/scripts/

`modelos/pedidos-do-aviso.md`

O pedido da aula com o porquê de cada parte, a versão para Telegram, Slack e Discord, e as continuações: o trap de saída que o próprio Claude sugeriu e a versão com menos mensagens.

No repositório: modulo-5-loop/aula-5.3-avisa-quando-termina-ou-trava/modelos/

`modelos/aviso.env.exemplo`

O arquivo de segredos comentado, canal por canal. Ao lado, o `ENSAIO-AVISO.md` (primeiro run de uma task, passa no `confere-phases.sh`) e o `run-morto.tsv` (faz o vigia avisar sem gastar token).

No repositório: modulo-5-loop/aula-5.3-avisa-quando-termina-ou-trava/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
O ralph rodou pelo Codex, ou "codex CLI nao encontrado"	O ralph da aula usa `--engine codex` por padrão. Com o Codex instalado, o run sai por ele, e gasta lá	Sempre `--engine claude`. O `loop.sh` já passa
"Notificacao WhatsApp desligada: ... ausentes em .../.env"	O link `.claude/scripts/.env` não existe, aponta para outro lugar, ou falta uma das três variáveis	`ls -l .claude/scripts/.env` e preencha URL, chave e instância
"nao foi possivel resolver o numero da instancia"	Chave ou nome da instância errados. Com o token da instância, depende da configuração do servidor (não testado)	Preencha `EVOLUTION_NUMBER` (só dígitos, com DDI e DDD)
"Falha ao enviar notificacao WhatsApp (HTTP 401)" ou "(HTTP 000)"	401: chave recusada. 000: servidor fora do ar ou URL errada	Acerte com o `aviso-env.sh` (passo 5). O run segue mesmo assim

SINTOMA	CAUSA	CORREÇÃO
"Arvore de trabalho suja" com `??` `.claude/scripts/.env`	O `.env` não está ignorado	`instalar.sh 5.3` traz o `.gitignore`; confira com `git check-ignore -v`
`git check-ignore` não mostra nada (passo 4)	O `.env` já foi commitado: o git não ignora arquivo rastreado	`git rm --cached .claude/scripts/.env`, commite e troque o token: ele ficou no histórico
Telegram: nada chega	Bot não começa conversa	Mande `/start` ao bot; o `chat_id` sai do `getUpdates`
Discord recusa o aviso	O `AVISO_WEBHOOK` manda `texto` e `text`; o webhook comum do Discord quer `content`	Use `SLACK_WEBHOOK_URL` com a URL do Discord e `/slack` no fim
O `loop.sh` não manda o WhatsApp	Sem `AVISO_CMD`, ele chama o `aviso.sh`, que não lê o arquivo	`export AVISO_CMD=.claude/scripts/aviso-env.sh` no mesmo terminal, na raiz
`NÃO hook guarda-aceite.py` no `--checa`	O hook da aula 4.2 não está instalado ou registrado	`instalar.sh 4.2` e preencha o `.claude/aceite.txt`
O "travou?" demora	`LOOP_QUIETO` é 15m e o vigia lê a cada 60 s	Para ver acontecer: `LOOP_QUIETO=1m` `LOOP_VIGIA_S=10`

6 Você sai *com*

- [ ] `git check-ignore -v .claude/scripts/.env` aponta para o `.gitignore` da aula, e o token mora em `~/.config/harness-lab/aviso.env` com `chmod 600`.
- [ ] `.claude/scripts/aviso-env.sh "teste"` chega no seu celular, pelo canal que você escolheu.
- [ ] O vigia avisou "morto?" com o `run-morto.tsv`, sem gastar token.
- [ ] O `loop.sh --checa ENSAIO-AVISO.md` passou e o ENSAIO de uma task terminou com o aviso no celular.
- [ ] O ralph da aula commitado em `.claude/scripts/`, pronto para rodar com `--engine claude`.

PROMETIDO NA AULA

- ✓ **"Vocês vão encontrar também o Ralph atualizado aí no material"**: `.claude/scripts/ralph.sh`, instalado no passo 3 (p. 5); no repositório, em `modulo-5-loop/aula-5.3-avisa-quando-termina-ou-trava/projeto/.claude/scripts/ralph.sh`.
- ✓ **"Eu coloquei um .env aqui com as credenciais"**: `modelos/aviso.env.exemplo`, com token, instância e URL, ligado no passo 4 (p. 5 e 6).
- ✓ **"Vocês podem usar bot do Telegram, Slack, Discord"**: o aviso de fim e de travou no seu canal pelo `aviso.sh` e pelo `aviso-env.sh` (passos 4 e 5, p. 5 e 6); o progresso fase a fase no seu canal: pedido 2 de `modelos/pedidos-do-aviso.md`; o Discord pelo endpoint `/slack` (seção 2, p. 3).
- ✓ **"O mais importante é a gente saber pedir"**: o pedido da aula, parte por parte, em `modelos/pedidos-do-aviso.md` (seção 1, p. 2).
- ✓ **"Posso colocar para rodar e ir na academia"**: o ENSAIO pelo `loop.sh` com o vigia, passo 7 (p. 7).

7 Glossário

Evolution API	Servidor que você hospeda e que conecta um número de WhatsApp a uma API REST. Não oficial no modo Baileys.
Instância	Um número conectado na Evolution, com nome (`EVOLUTION_INSTANCE`) e token próprio. O dono dela (`ownerJid`) é o destino quando `EVOLUTION_NUMBER` está vazio.
run.tsv	`.phases/state/run.tsv`: o estado que o ralph do plugin publica (status, PID, fase, ciclo, gate, último erro).
Vigia	O `vigia.sh`: lê o `run.tsv` de fora e avisa o que o ralph não sabe dizer (travou, morreu).
AVISO_CMD	Troca quem manda a mensagem no `loop.sh` e no vigia.
ENSAIO	Um PHASES de uma fase e uma task, numa branch descartável, para ver o loop rodar com o CLI real antes da feature.

8 Para *ir além*

- Automate actions with hooks: "Limitations", por que o Stop não é "terminou", e o `StopFailure` para limite de API.
- Telegram Bot API: `sendMessage`, `getUpdates` e o limite de 4096 caracteres por mensagem.
- Discord Webhook Resource: "Execute Slack-Compatible Webhook" e o que ele não suporta.
- Evolution API · Send Text Message: o endpoint que o ralph da aula chama e a autenticação por `apikey`.
- WhatsApp Business Platform · Pricing: a cobrança por mensagem, para quem quer o WhatsApp oficial.

Próxima: *Aula 6.1 · Rodando a feature-alvo*

Módulo 6: o loop roda a feature-alvo escolhida no começo do curso, do PHASES ao último commit, com o celular avisando.